

Python Interview Questions

Iterators, Generators & yield — Top 100

Protocol · yield · send() · yield from · async · itertools · Patterns

Section 1 — The Iterator Protocol (Q1–Q20)

Q1. What is the iterator protocol in Python?

The iterator protocol requires an object to implement two methods: `__iter__()` which returns the iterator itself, and `__next__()` which returns the next value or raises `StopIteration` when exhausted. Any object implementing both is an iterator.

```
class Counter:
    def __init__(self, start, stop):
        self.current = start
        self.stop = stop

    def __iter__(self):
        # makes it an iterable
        return self          # returns itself – it IS the iterator

    def __next__(self):
        # advances and returns next value
        if self.current >= self.stop:
            raise StopIteration
        val = self.current
        self.current += 1
        return val

for n in Counter(1, 4):
    print(n)    # 1 2 3
```

Q2. What is the difference between an iterable and an iterator?

An iterable has `__iter__()` that returns a fresh iterator each time it is called. An iterator has `__next__()` and `__iter__()` (returning self). Iterators are stateful and single-pass; iterables can produce multiple independent iterators.

```
lst = [1, 2, 3]          # iterable – not an iterator
it = iter(lst)           # iterator produced from the iterable

print(next(it))  # 1
print(next(it))  # 2

# A new iterator starts from the beginning
it2 = iter(lst)
```

```
print(next(it2)) # 1

# The iterator itself is an iterable (returns self)
print(iter(it) is it) # True
```

Q3. What does `iter()` do internally?

`iter(obj)` calls `obj.__iter__()`. With two arguments, `iter(callable, sentinel)` creates an iterator that calls `callable()` repeatedly until the result equals `sentinel` — useful for reading chunks until EOF.

```
# Single-arg form
it = iter([10, 20, 30])
print(next(it)) # 10

# Two-arg form: iter(callable, sentinel)
import io
stream = io.StringIO('line1\nline2\n')
for chunk in iter(stream.readline, ''):
    print(repr(chunk)) # 'line1\n' 'line2\n'
```

Q4. What does `next()` do and what is its optional default argument?

`next(it)` calls `it.__next__()`. If `StopIteration` would be raised and a default is supplied, `next()` returns the default instead of propagating the exception.

```
it = iter([1, 2])
print(next(it)) # 1
print(next(it)) # 2
print(next(it, 'DONE')) # 'DONE' — no exception

# Without default:
# next(it) # StopIteration
```

Q5. What happens when `StopIteration` is raised inside a for loop?

The `for` statement catches `StopIteration` silently and terminates the loop. This is the standard protocol — you should never raise `StopIteration` yourself in a generator (use `return` instead).

```
class Once:
    def __init__(self, val): self.val = val; self.done = False
    def __iter__(self): return self
    def __next__(self):
        if self.done: raise StopIteration
        self.done = True
        return self.val

for x in Once(42):
    print(x) # 42 — loop exits cleanly after one item
```

Q6. What is the difference between `__iter__` and `__getitem__` for iteration?

Python also supports the legacy sequence protocol: if a class defines `__getitem__` with integer indices starting from 0, `iter()` wraps it in an iterator automatically, raising `IndexError` to stop. `__iter__` takes priority when both are defined.

```
class OldSeq:
    def __getitem__(self, index):
        if index >= 3: raise IndexError
        return index * 10

for x in OldSeq():
    print(x)    # 0  10  20

# Works without __iter__ — legacy protocol
print(list(OldSeq()))    # [0, 10, 20]
```

Q7. Can you reuse an exhausted iterator?

No. Once `__next__()` raises `StopIteration`, the iterator is exhausted. Calling `next()` again returns `StopIteration` again (by contract). To re-iterate, create a new iterator from the original iterable.

```
lst = [1, 2, 3]
it = iter(lst)
print(list(it))    # [1, 2, 3]
print(list(it))    # [] — exhausted

# Reset by creating a new iterator
it = iter(lst)
print(list(it))    # [1, 2, 3]
```

Q8. How does Python's for loop work under the hood?

The for loop calls `iter()` to get an iterator, then repeatedly calls `next()` until `StopIteration` is caught. It is equivalent to a while loop using these two primitives.

```
items = [10, 20, 30]

# What 'for x in items: print(x)' actually does:
_iter = iter(items)
while True:
    try:
        x = next(_iter)
    except StopIteration:
        break
    print(x)    # 10  20  30
```

Q9. What is a lazy iterator and why is it useful?

A lazy iterator produces values on demand rather than building the entire sequence in memory. This is critical for large or infinite sequences — it keeps memory usage $O(1)$ regardless of the logical size of the sequence.

```
# Eager – builds full list in memory
squares_eager = [x**2 for x in range(10**7)]    # ~80 MB

# Lazy – produces one value at a time
squares_lazy = (x**2 for x in range(10**7))    # ~200 bytes

import sys
print(sys.getsizeof(squares_lazy))    # 104 or 200 bytes
```

Q10. What is the itertools module and name five essential functions?

itertools provides fast, memory-efficient iterator building blocks. Five essential ones: count (infinite counter), chain (concatenate iterables), islice (slice an iterator), groupby (group consecutive elements), product (cartesian product).

```
import itertools

# count – infinite
for n in itertools.islice(itertools.count(10, 2), 4):
    print(n, end=' ')    # 10 12 14 16

# chain
print(list(itertools.chain([1,2], [3,4], [5])))    # [1,2,3,4,5]

# groupby
data = [('a',1),('a',2),('b',3)]
for k, g in itertools.groupby(data, key=lambda x: x[0]):
    print(k, list(g))
```

Q11. What is enumerate() and how is it implemented?

enumerate(iterable, start=0) wraps an iterable to produce (index, value) pairs. It is itself a lazy iterator. You can replicate it with a generator.

```
fruits = ['apple', 'banana', 'cherry']
for i, f in enumerate(fruits, start=1):
    print(i, f)
# 1 apple 2 banana 3 cherry

# Manual equivalent
def my_enumerate(iterable, start=0):
    n = start
    for item in iterable:
        yield n, item
        n += 1
```

Q12. What is zip() and how does it handle iterables of different lengths?

zip() pairs elements from multiple iterables, stopping at the shortest. Use itertools.zip_longest() to pad shorter iterables with a fill value.

```
a = [1, 2, 3]
b = ['x', 'y']

print(list(zip(a, b)))          # [(1, 'x'), (2, 'y')]

from itertools import zip_longest
print(list(zip_longest(a, b, fillvalue=0)))
# [(1, 'x'), (2, 'y'), (3, 0)]
```

Q13. What is map() and how does it relate to iterators?

map(func, iterable) returns a lazy iterator that applies func to each element. It does not build a list — values are computed on demand.

```
result = map(str.upper, ['hello', 'world'])
print(type(result))          # <class 'map'>
print(next(result))          # HELLO
print(list(result))          # ['WORLD']

# With multiple iterables
sums = map(lambda x, y: x + y, [1, 2, 3], [10, 20, 30])
print(list(sums))            # [11, 22, 33]
```

Q14. What is filter() and how does it work?

filter(func, iterable) returns a lazy iterator of elements for which func returns truthy. If func is None, falsy elements are removed.

```
evens = filter(lambda x: x % 2 == 0, range(10))
print(list(evens))           # [0, 2, 4, 6, 8]

# func=None removes falsy values
data = [0, 1, '', 'hi', None, 42]
print(list(filter(None, data))) # [1, 'hi', 42]
```

Q15. What is the difference between a list comprehension and a generator expression?

A list comprehension uses [] and eagerly builds a list in memory. A generator expression uses () and is a lazy iterator — it yields values on demand without materialising the collection.

```
# List comprehension – O(n) memory
lc = [x**2 for x in range(5)]
print(type(lc), lc)          # <class 'list'> [0, 1, 4, 9, 16]

# Generator expression – O(1) memory
```

```
ge = (x**2 for x in range(5))
print(type(ge))      # <class 'generator'>
print(next(ge))      # 0
print(list(ge))      # [1, 4, 9, 16] - 0 already consumed
```

Q16. How do you implement an infinite iterator?

Omit the `StopIteration` condition in `__next__`, or use `itertools.count()`. Always pair with a termination mechanism like `islice` or a `break` condition.

```
class Naturals:
    def __init__(self): self.n = 0
    def __iter__(self): return self
    def __next__(self):
        self.n += 1
        return self.n

from itertools import islice
print(list(islice(Naturals(), 5))) # [1, 2, 3, 4, 5]

# Or with itertools.count
import itertools
print(list(islice(itertools.count(1), 5))) # [1, 2, 3, 4, 5]
```

Q17. What is the `__length_hint__` method?

`__length_hint__()` provides an estimated (possibly incorrect) length for an iterator, used by `list()` and similar to pre-allocate memory. It is optional and separate from `__len__`.

```
import operator

lst = [1, 2, 3, 4, 5]
it = iter(lst)
print(operator.length_hint(it)) # 5
next(it)
print(operator.length_hint(it)) # 4 - hint decreases

# Implement in custom iterator
class MyIter:
    def __init__(self, n): self.n = n; self.i = 0
    def __iter__(self): return self
    def __next__(self):
        if self.i >= self.n: raise StopIteration
        v = self.i; self.i += 1; return v
    def __length_hint__(self): return max(0, self.n - self.i)
```

Q18. What is `itertools.chain()` and when do you use it?

`chain(*iterables)` lazily concatenates multiple iterables end-to-end without building intermediate lists. Use `chain.from_iterable(iterable_of_iterables)` when the inputs are themselves an iterable.

```
import itertools

a, b, c = [1, 2], [3, 4], [5]
for x in itertools.chain(a, b, c):
    print(x, end=' ')    # 1 2 3 4 5

# from_iterable: flatten a list of lists
nested = [[1,2],[3,4],[5]]
flat = list(itertools.chain.from_iterable(nested))
print(flat)    # [1, 2, 3, 4, 5]
```

Q19. How does reversed() work and what does a class need to support it?

reversed(seq) requires either `__reversed__()` or both `__len__()` and `__getitem__()`. It returns a reverse iterator without copying the sequence.

```
lst = [1, 2, 3, 4, 5]
for x in reversed(lst):
    print(x, end=' ')    # 5 4 3 2 1

class Deck:
    def __init__(self, cards): self.cards = cards
    def __len__(self): return len(self.cards)
    def __getitem__(self, i): return self.cards[i]
    def __reversed__(self): return reversed(self.cards)

d = Deck(['A', 'K', 'Q'])
print(list(reversed(d)))    # ['Q', 'K', 'A']
```

Q20. What is itertools.tee() and what are its caveats?

tee(iterable, n=2) creates n independent iterators from one. Once tee() is called, do NOT advance the original iterator — all values must come through the tee'd copies. Memory grows proportionally to how far one copy is ahead of another.

```
import itertools

def gen():
    yield from range(5)

g = gen()
a, b = itertools.tee(g, 2)

print(list(a))    # [0, 1, 2, 3, 4]
print(list(b))    # [0, 1, 2, 3, 4] - independent copy

# WARNING: advancing g directly after tee() causes undefined behaviour
```

Section 2 — Generators & yield (Q21–Q50)

Q21. What is a generator function?

A generator function contains at least one `yield` statement. Calling it does not execute the body — it returns a generator object. The body runs lazily, suspending at each `yield` and resuming on the next `next()` call.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

gen = countdown(3)
print(type(gen))      # <class 'generator'>
print(next(gen))      # 3
print(next(gen))      # 2
print(next(gen))      # 1
# next(gen)           # StopIteration
```

Q22. What does `yield` do exactly?

`yield` suspends the function's execution frame, saves all local state (local variables, instruction pointer, stack), returns the yielded value to the caller, and waits until `next()` is called again to resume.

```
def trace():
    print('A')      # runs on first next()
    yield 1
    print('B')      # runs on second next()
    yield 2
    print('C')      # runs after second yield, before StopIteration

g = trace()
print(next(g))     # prints 'A', returns 1
print(next(g))     # prints 'B', returns 2
# next(g)          # prints 'C', raises StopIteration
```

Q23. What is the difference between `return` and `yield` in a generator?

`yield` produces a value and suspends; the generator can be resumed. `return` (or falling off the end) terminates the generator and optionally sets a value as the `StopIteration` exception's `value` attribute.

```
def gen():
    yield 1
    yield 2
    return 'done'    # becomes StopIteration.value

g = gen()
print(next(g))      # 1
print(next(g))      # 2
try:
    next(g)
except StopIteration as e:
```



```
print(e.value)    # 'done'
```

Q24. What is yield from?

yield from sub_iterable delegates iteration to a sub-generator or any iterable. It transparently passes values, send(), throw(), and close() through to the sub-generator and captures its return value.

```
def flatten(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)    # recurse
        else:
            yield item

data = [1, [2, [3, 4]], 5]
print(list(flatten(data)))    # [1, 2, 3, 4, 5]
```

Q25. How does yield from differ from a manual yield loop?

A manual 'for x in sub: yield x' does not propagate send(), throw(), or close() to the sub-generator, and does not capture its return value. yield from handles all of these automatically.

```
def sub():
    value = yield 'ready'
    yield f'got: {value}'

def delegator():
    result = yield from sub()    # propagates send()

g = delegator()
print(next(g))    # 'ready'
print(g.send('hi'))    # 'got: hi'
```

Q26. What is a generator expression?

A generator expression is a compact generator written inline with () syntax. It is syntactic sugar for a one-expression generator function and is lazily evaluated.

```
# Equivalent ways to produce squares lazily
def squares(n):
    for x in range(n):
        yield x ** 2

gen_expr = (x**2 for x in range(5))

print(list(squares(5)))    # [0, 1, 4, 9, 16]
print(list(gen_expr))    # [0, 1, 4, 9, 16]

# Generator expression inside a function call (no extra parens needed)
total = sum(x**2 for x in range(5))
```

```
print(total)    # 30
```

Q27. How do you send a value into a generator with send()?

`g.send(value)` resumes the generator and makes `yield` evaluate to `value` instead of `None`. Before the first `send()`, you must call `next()` or `send(None)` to advance to the first `yield`.

```
def accumulator():
    total = 0
    while True:
        value = yield total
        if value is None:
            break
        total += value

g = accumulator()
next(g)          # prime: advances to first yield, returns 0
print(g.send(10)) # 10
print(g.send(20)) # 30
print(g.send(5))  # 35
```

Q28. What is `g.throw()` used for?

`g.throw(ExcType, value, traceback)` injects an exception at the point where the generator is suspended. The generator can catch it with `try/except` inside itself.

```
def safe_gen():
    try:
        while True:
            yield
    except ValueError as e:
        print(f'Caught inside generator: {e}')
        yield 'recovered'

g = safe_gen()
next(g)
print(g.throw(ValueError, 'bad input')) # Caught... 'recovered'
```

Q29. What is `g.close()` and how does it work?

`g.close()` throws `GeneratorExit` at the suspension point. If the generator catches it and yields again, `RuntimeError` is raised. Generators should clean up in a `finally` block, not catch `GeneratorExit` and continue.

```
def managed():
    try:
        yield 1
        yield 2
    finally:
        print('cleanup!') # runs on close()
```

```
g = managed()
print(next(g))    # 1
g.close()         # prints 'cleanup!'
```

Q30. What is StopIteration and when does a generator raise it?

StopIteration is raised by `__next__()` to signal exhaustion. In a generator, it is raised automatically when the function body returns (explicitly or by falling off the end). Its value attribute carries the return value if any.

```
def gen():
    yield 1
    # implicit return – raises StopIteration

g = gen()
print(next(g))    # 1
try:
    next(g)
except StopIteration as e:
    print('Stopped, value =', e.value)    # None
```

Q31. Why should you NOT raise StopIteration inside a generator?

PEP 479 (Python 3.7+) converts StopIteration raised inside a generator into RuntimeError. This prevents subtle bugs where an inner StopIteration accidentally terminates an outer generator. Use return instead.

```
def buggy():
    raise StopIteration    # RuntimeError in Python 3.7+
    yield

def correct():
    return                # properly terminates generator
    yield

import sys
g = buggy()
try:
    next(g)
except RuntimeError as e:
    print(e)    # generator raised StopIteration
```

Q32. How do generators save memory compared to lists?

A generator holds only the current execution frame (a fixed amount of state), regardless of how many values it will eventually produce. A list holds all values simultaneously. For large or infinite sequences the difference is dramatic.

```
import sys
```

```

N = 10_000_000

# List – ~85 MB for 10M ints
# lst = list(range(N))
# print(sys.getsizeof(lst))

# Generator – ~208 bytes always
gen = (x for x in range(N))
print(sys.getsizeof(gen)) # 104 or 208 bytes

# Sum without building list
total = sum(x for x in range(N))
print(total) # 49999995000000

```

Q33. How do you create a pipeline of generators?

Chain generators so each feeds into the next. Values flow lazily through the pipeline — no intermediate collection is materialised.

```

def read_lines(path):
    with open(path) as f:
        yield from f

def strip(lines):
    for line in lines:
        yield line.strip()

def non_empty(lines):
    for line in lines:
        if line:
            yield line

def to_upper(lines):
    for line in lines:
        yield line.upper()

# Compose the pipeline – all lazy
# pipeline = to_upper(non_empty(strip(read_lines('data.txt'))))
# for line in pipeline: process(line)

```

Q34. What is a coroutine and how does it relate to generators?

A coroutine is a generator that both produces values (yield) and receives values (send()). Classic coroutines are generator functions that primarily use yield as an expression to receive data, not just to produce it.

```

def grep(pattern):
    print(f'Searching for: {pattern}')
    while True:
        line = yield # receive data via send()
        if pattern in line:
            print(line)

```

```
g = grep('error')
next(g)           # prime
g.send('info: ok')
g.send('error: boom') # prints 'error: boom'
g.send('debug: fine')
```

Q35. What is the difference between `async def` and a generator?

A generator function uses `yield`; calling it returns a generator object driven by `next()`. An `async def` function uses `await`; calling it returns a coroutine object driven by an event loop (`asyncio`). Async generators combine both: `async def` + `yield`.

```
# Generator
def gen(): yield 1; yield 2
g = gen(); print(next(g)) # 1

# Coroutine (async)
import asyncio
async def coro(): return 42
print(asyncio.run(coro())) # 42

# Async generator
async def agen():
    yield 1; yield 2
async def consume():
    async for x in agen(): print(x)
asyncio.run(consume()) # 1 2
```

Q36. What is `itertools.islice()` and why is it important for generators?

`islice(iterable, stop)` or `islice(iterable, start, stop, step)` lazily slices any iterator without materialising it — critical for infinite generators.

```
import itertools

def naturals():
    n = 1
    while True:
        yield n
        n += 1

first_10 = list(itertools.islice(naturals(), 10))
print(first_10) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Slice from 5 to 15
print(list(itertools.islice(naturals(), 5, 15)))
# [6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

Q37. What is `itertools.takewhile()` and `dropwhile()`?

`takewhile(pred, it)` yields elements while `pred` is `True`, then stops. `dropwhile(pred, it)` skips elements while `pred` is `True`, then yields the rest.

```
import itertools

data = [2, 4, 6, 7, 8, 10]

print(list(itertools.takewhile(lambda x: x % 2 == 0, data)))
# [2, 4, 6] – stops at 7

print(list(itertools.dropwhile(lambda x: x % 2 == 0, data)))
# [7, 8, 10] – drops until first odd, then yields rest
```

Q38. What is `itertools.groupby()`?

`groupby(iterable, key)` groups consecutive elements with the same key. The input must be sorted by key for logical grouping. Each group is itself a lazy iterator.

```
import itertools

data = [
    {'dept': 'Eng', 'name': 'Alice'},
    {'dept': 'Eng', 'name': 'Bob'},
    {'dept': 'HR', 'name': 'Carol'},
]

for dept, members in itertools.groupby(data, key=lambda r: r['dept']):
    print(dept, [m['name'] for m in members])
# Eng ['Alice', 'Bob']
# HR  ['Carol']
```

Q39. What is `itertools.product()`?

`product(*iterables, repeat=1)` computes the cartesian product lazily, equivalent to nested for loops.

```
import itertools

for pair in itertools.product('AB', [1, 2]):
    print(pair, end=' ') # ('A',1) ('A',2) ('B',1) ('B',2)

# repeat – same iterable multiple times
print(list(itertools.product(range(2), repeat=3)))
# all 8 binary triplets: (0,0,0),(0,0,1),...,(1,1,1)
```

Q40. What is `itertools.accumulate()`?

`accumulate(iterable, func=operator.add, initial=None)` produces running totals (or any running aggregation). Added `initial=` in Python 3.8.

```
import itertools, operator
```

```
data = [1, 2, 3, 4, 5]

print(list(itertools.accumulate(data)))
# [1, 3, 6, 10, 15] – running sum

print(list(itertools.accumulate(data, operator.mul)))
# [1, 2, 6, 24, 120] – running product (factorial)

print(list(itertools.accumulate(data, initial=100)))
# [100, 101, 103, 106, 110, 115]
```

Q41. How do you implement a Fibonacci generator?

Use a generator function with a while True loop and local state — clean, memory-efficient, and infinite.

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

from itertools import islice
print(list(islice(fibonacci(), 10)))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Q42. How do you use a generator to read a large file line by line?

File objects in Python are already iterators. A generator wrapper can add pre-processing (strip, filter). This keeps only one line in memory at a time.

```
def read_csv_rows(path):
    with open(path, encoding='utf-8') as f:
        header = next(f).strip().split(',')
        for line in f:
            values = line.strip().split(',')
            yield dict(zip(header, values))

# for row in read_csv_rows('big.csv'):
#     process(row)           # O(1) memory
```

Q43. What is a bidirectional generator / coroutine pattern?

A generator that both yields data (output) and receives data via send() (input) acts as a coroutine — a cooperative, bidirectional channel.

```
def running_average():
    total = count = 0
    average = None
    while True:
```

```

        value = yield average
        if value is None:
            break
        total += value
        count += 1
        average = total / count

ra = running_average()
next(ra)           # prime
print(ra.send(10)) # 10.0
print(ra.send(20)) # 15.0
print(ra.send(30)) # 20.0

```

Q44. How do you flatten a nested structure with a generator?

Use a recursive generator with yield from for nested lists and yield for scalar values.

```

def deep_flatten(iterable):
    for item in iterable:
        try:
            iter(item)           # is it iterable?
            if isinstance(item, (str, bytes)):
                yield item       # treat str as scalar
            else:
                yield from deep_flatten(item)
        except TypeError:
            yield item

nested = [1, [2, [3, (4, 5)]], 6]
print(list(deep_flatten(nested))) # [1, 2, 3, 4, 5, 6]

```

Q45. What is `itertools.chain.from_iterable()` and how is it different from `chain()`?

`chain(*iterables)` takes variadic args; `chain.from_iterable(iterable_of_iterables)` takes a single lazy iterable of iterables — useful when you don't know all sub-iterables upfront.

```

import itertools

# chain() – variadic args
print(list(itertools.chain([1,2],[3,4]))) # [1, 2, 3, 4]

# from_iterable – lazy generator of generators
def gen_of_gens():
    yield range(1, 3)
    yield range(3, 5)

print(list(itertools.chain.from_iterable(gen_of_gens())))
# [1, 2, 3, 4]

```

Q46. How do you use generators for data transformation pipelines?

Define each transformation as a generator that takes an iterable and yields transformed values. Compose them by nesting — data flows lazily through without intermediate storage.

```
import csv, io

raw = 'name,score\nAlice,90\nBob,85\nCarol,95'

def parse_csv(text):
    reader = csv.DictReader(io.StringIO(text))
    yield from reader

def to_int_scores(rows):
    for row in rows:
        row['score'] = int(row['score'])
        yield row

def above(rows, threshold):
    for row in rows:
        if row['score'] >= threshold:
            yield row

pipeline = above(to_int_scores(parse_csv(raw)), 88)
for r in pipeline:
    print(r)    # Alice 90, Carol 95
```

Q47. What is the return value of a generator function used with yield from?

The value returned by the sub-generator's return statement is captured as the result of the yield from expression in the delegating generator.

```
def sub():
    yield 1
    yield 2
    return 'sub_done'

def delegator():
    result = yield from sub()
    print('Sub returned:', result)    # 'sub_done'
    yield 3

for x in delegator():
    print(x)    # 1 2 (then 'Sub returned: sub_done') 3
```

Q48. How do you implement a generator-based state machine?

Each state is a generator function. yield suspends in the current state; send() drives transitions.

```
def traffic_light():
    while True:
        yield 'GREEN'
        yield 'YELLOW'
        yield 'RED'

light = traffic_light()
```

```
for _ in range(6):
    print(next(light), end=' ')
# GREEN YELLOW RED GREEN YELLOW RED
```

Q49. How do generators interact with try/finally?

finally blocks in a generator execute when the generator is closed (via `g.close()`), garbage collection, or falling out of scope). This makes generators safe for resource management.

```
def resource_gen():
    print('Acquiring resource')
    try:
        yield 1
        yield 2
        yield 3
    finally:
        print('Releasing resource') # always runs

g = resource_gen()
print(next(g)) # Acquiring resource 1
print(next(g)) # 2
g.close()      # Releasing resource
```

Q50. What is the `gi_frame`, `gi_code`, `gi_running` attribute on a generator?

These are internal attributes of the generator object exposing its current state: `gi_frame` is the execution frame (locals, line number), `gi_code` is the code object, `gi_running` is `True` while the generator is currently executing.

```
def gen():
    x = 42
    yield x

g = gen()
next(g)
print(g.gi_frame.f_locals) # {'x': 42}
print(g.gi_code.co_name)   # 'gen'
print(g.gi_running)        # False (suspended, not running)
```

Section 3 — Async Generators & Advanced Patterns (Q51–Q75)

Q51. What is an async generator?

An async generator is an async def function containing yield. It produces values asynchronously and is consumed with async for. It supports `__aiter__()` and `__anext__()`.

```
import asyncio

async def async_range(n):
    for i in range(n):
        await asyncio.sleep(0)
        yield i

async def main():
    async for x in async_range(3):
        print(x)    # 0 1 2

asyncio.run(main())
```

Q52. What is `__aiter__` and `__anext__`?

`__aiter__()` returns the async iterator object (usually self). `__anext__()` is an async method returning the next value or raising `StopAsyncIteration`. Together they form the async iterator protocol.

```
import asyncio

class AsyncCounter:
    def __init__(self, n): self.n = n; self.i = 0
    def __aiter__(self): return self
    async def __anext__(self):
        if self.i >= self.n:
            raise StopAsyncIteration
        await asyncio.sleep(0)
        val = self.i; self.i += 1
        return val

async def main():
    async for x in AsyncCounter(3):
        print(x)    # 0 1 2

asyncio.run(main())
```

Q53. How do you close an async generator?

Call `aclose()` which is a coroutine. It throws `GeneratorExit` into the async generator and awaits any finally cleanup. Use `async with contextlib.aclosing()` for automatic cleanup.

```
import asyncio

async def agen():
    try:
        yield 1; yield 2
    finally:
        print('async cleanup')

async def main():
    g = agen()
```

```
print(await g.__anext__())    # 1
await g.aclose()             # 'async cleanup'

asyncio.run(main())
```

Q54. What is `contextlib.contextmanager` and how does it use generators?

`@contextmanager` turns a generator function with exactly one `yield` into a context manager. Code before `yield` runs on `__enter__`; code after `yield` runs on `__exit__`. Exceptions are thrown into the generator.

```
from contextlib import contextmanager
import time

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                # __enter__ returns here
    finally:
        elapsed = time.perf_counter() - start
        print(f'{label}: {elapsed:.4f}s')

with timer('sort'):
    sorted(range(100_000))
```

Q55. What is `contextlib.asynccontextmanager`?

Like `@contextmanager` but for `async def` generators — wraps an `async` generator into an `async` context manager usable with `async with`.

```
from contextlib import asynccontextmanager
import asyncio

@asynccontextmanager
async def managed_connection(host):
    print(f'Connecting to {host}')
    try:
        yield {'host': host, 'active': True}
    finally:
        print(f'Disconnecting from {host}')

async def main():
    async with managed_connection('db.local') as conn:
        print('Using', conn)

asyncio.run(main())
```

Q56. What is `itertools.cycle()` and what should you watch out for?

`cycle(iterable)` repeats an iterable indefinitely. It stores a copy of each element the first time through, so it does use memory proportional to the iterable length.

```
import itertools

colours = itertools.cycle(['red', 'green', 'blue'])
for _, colour in zip(range(7), colours):
    print(colour, end=' ')
# red green blue red green blue red

# Watch out: cycle stores all elements after first pass
# Do not cycle an infinite generator!
```

Q57. What is `itertools.repeat()`?

`repeat(obj, times=None)` yields the same object times times (or infinitely). Useful as a constant source in `zip` or `map` operations.

```
import itertools

# Fixed repetition
print(list(itertools.repeat(42, 3))) # [42, 42, 42]

# Use with map as a constant
result = list(map(pow, range(5), itertools.repeat(2)))
print(result) # [0, 1, 4, 9, 16] - squares
```

Q58. What is `itertools.starmap()`?

`starmap(func, iterable)` applies `func` to each element of `iterable` using `*` unpacking. Unlike `map()`, it expects each element to be a tuple of arguments.

```
import itertools

pairs = [(2, 3), (4, 2), (10, 3)]
result = list(itertools.starmap(pow, pairs))
print(result) # [8, 16, 1000]

# Equivalent to: [pow(*args) for args in pairs]
```

Q59. What is `itertools.combinations()` and `permutations()`?

`combinations(iterable, r)` yields `r`-length tuples in sorted order, no repetition. `permutations(iterable, r)` yields all orderings.

```
import itertools

print(list(itertools.combinations('ABC', 2)))
# [('A', 'B'), ('A', 'C'), ('B', 'C')]
```

```
print(list(itertools.permutations('AB', 2)))
# [('A', 'B'), ('B', 'A')]

print(list(itertools.combinations_with_replacement('AB', 2)))
# [('A', 'A'), ('A', 'B'), ('B', 'B')]
```

Q60. How do you implement lazy pagination with a generator?

Yield pages of results, fetching the next page only when the consumer asks — ideal for API pagination.

```
def paginate(fetch_fn, page_size=10):
    page = 1
    while True:
        results = fetch_fn(page=page, size=page_size)
        if not results:
            break
        yield from results
        if len(results) < page_size:
            break
        page += 1

# Example stub
db = list(range(25))
def fetch(page, size): return db[(page-1)*size : page*size]

print(list(paginate(fetch, 10))) # [0..24]
```

Q61. What is `functools.reduce()` and how does it relate to iterators?

`reduce(func, iterable, initializer)` applies `func` cumulatively to an iterator, producing a single result. It consumes the iterator left-to-right.

```
from functools import reduce

# Sum
print(reduce(lambda acc, x: acc + x, range(1, 6))) # 15

# Product
print(reduce(lambda acc, x: acc * x, range(1, 6), 1)) # 120

# Build dict from pairs
pairs = [('a',1),('b',2),('c',3)]
d = reduce(lambda acc, kv: {**acc, kv[0]: kv[1]}, pairs, {})
print(d) # {'a':1,'b':2,'c':3}
```

Q62. How do you merge sorted iterators lazily?

`heapq.merge(*iterables)` lazily merges pre-sorted iterables into a single sorted stream without loading everything into memory.

```
import heapq

a = [1, 4, 7]
b = [2, 5, 8]
c = [3, 6, 9]

merged = heapq.merge(a, b, c)
print(list(merged))  # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Q63. What is the difference between `iter()` one-arg and two-arg forms?

One-arg: `iter(obj)` calls `obj.__iter__()`. Two-arg: `iter(callable, sentinel)` creates a `CallableIterator` that calls `callable()` repeatedly until the return value `== sentinel`, then raises `StopIteration`.

```
import struct

# Read 4-byte blocks until empty bytes
data = b'\x01\x02\x03\x04\x05\x06\x07\x08'
import io
buf = io.BytesIO(data)

blocks = list(iter(lambda: buf.read(4), b''))
print(blocks)  # [b'\x01\x02\x03\x04', b'\x05\x06\x07\x08']
```

Q64. How do you implement a sliding window with generators?

Combine a deque with `maxlen` and yield a snapshot of it at each step — $O(1)$ memory per window.

```
from collections import deque

def sliding_window(iterable, n):
    it = iter(iterable)
    win = deque(maxlen=n)
    for _ in range(n):  # fill initial window
        win.append(next(it))
    yield tuple(win)
    for item in it:  # slide
        win.append(item)
        yield tuple(win)

data = range(6)
print(list(sliding_window(data, 3)))
# [(0,1,2), (1,2,3), (2,3,4), (3,4,5)]
```

Q65. How do you build a round-robin iterator over multiple iterables?

`itertools.chain` and `cycle` help, but the classic approach uses a deque of iterators — when one is exhausted it is removed.

```
from collections import deque
```

```
def roundrobin(*iterables):
    pending = deque(map(iter, iterables))
    while pending:
        it = pending.popleft()
        try:
            yield next(it)
            pending.append(it)
        except StopIteration:
            pass # drop exhausted iterator

print(list(roundrobin('ABC', [1,2], ['x'])))
# ['A', 1, 'x', 'B', 2, 'C']
```

Q66. What is `itertools.zip_longest()` and when do you use it?

`zip_longest(*iterables, fillvalue=None)` zips iterables of unequal length, padding shorter ones with `fillvalue`. Standard `zip` stops at the shortest.

```
from itertools import zip_longest

a = [1, 2, 3]
b = ['a', 'b']

print(list(zip(a, b)))           # [(1, 'a'), (2, 'b')]
print(list(zip_longest(a, b, fillvalue=0))) # [(1, 'a'), (2, 'b'), (3, 0)]
```

Q67. What is `itertools.compress()`?

`compress(data, selectors)` filters data, keeping only elements where the corresponding selector is truthy. Both are consumed in lockstep.

```
import itertools

data      = ['a', 'b', 'c', 'd', 'e']
selectors = [1, 0, 1, 0, 1]

print(list(itertools.compress(data, selectors)))
# ['a', 'c', 'e']
```

Q68. How do you implement a peek-able iterator?

Wrap any iterator to allow looking at the next item without consuming it, using a one-item buffer.

```
class Peekable:
    def __init__(self, iterable):
        self._it = iter(iterable)
        self._peek = None
        self._has = False
        self._advance()
```



```

def _advance(self):
    try:
        self._peek = next(self._it)
        self._has = True
    except StopIteration:
        self._has = False

def peek(self): return self._peek if self._has else None
def __iter__(self): return self
def __next__(self):
    if not self._has: raise StopIteration
    val = self._peek
    self._advance()
    return val

p = Peekable([1, 2, 3])
print(p.peak())    # 1 (not consumed)
print(next(p))     # 1
print(p.peak())    # 2

```

Q69. How do generators enable lazy evaluation of expensive operations?

Because values are produced only when requested, expensive computations are deferred. If the consumer stops early (via `break` or `islice`), never-needed values are never computed.

```

def expensive_transform(items):
    for item in items:
        result = item ** 2    # imagine this is costly
        yield result

data = range(10_000_000)
gen = expensive_transform(data)

# Only first 5 computed
from itertools import islice
print(list(islice(gen, 5)))  # [0, 1, 4, 9, 16]

```

Q70. What is the difference between `send(None)` and `next()` on a generator?

They are functionally identical — both advance the generator one step. `send(None)` is used when you want to prime a coroutine-style generator without a separate `next()` call, and is slightly more explicit in intent.

```

def echo():
    while True:
        received = yield
        print('Got:', received)

g = echo()
g.send(None)    # prime (same as next(g))
g.send('hello') # Got: hello
g.send('world') # Got: world

```

Q71. What is a delegating generator?

A delegating generator uses `yield from` to pass control to a sub-generator, transparently forwarding `send()`, `throw()`, and `close()` through itself.

```
def inner():
    x = yield 'inner_1'
    yield f'inner_2 got {x}'

def outer():
    yield 'outer_start'
    yield from inner()          # delegates to inner
    yield 'outer_end'

g = outer()
print(next(g))                 # 'outer_start'
print(next(g))                 # 'inner_1'
print(g.send('hi'))             # 'inner_2 got hi'
print(next(g))                 # 'outer_end'
```

Q72. How can generators implement cooperative multitasking?

A simple scheduler can drive multiple generators, switching between them using `send()/next()`. This is the conceptual basis of Python's `asyncio` event loop.

```
from collections import deque

def task(name, n):
    for i in range(n):
        print(f'{name}: step {i}')
        yield                    # voluntarily yield CPU

def scheduler(*tasks):
    queue = deque(tasks)
    while queue:
        t = queue.popleft()
        try:
            next(t)
            queue.append(t)        # reschedule if not done
        except StopIteration:
            pass

scheduler(task('A', 3), task('B', 2))
# A:0 B:0 A:1 B:1 A:2
```

Q73. How do you convert a callback-based API to a generator?

Use a queue inside a thread: the callback pushes items onto the queue; the generator pops them. A sentinel value signals completion.

```
from queue import Queue
import threading
```

```
def callback_to_generator(start_fn):
    q = Queue()
    DONE = object()

    def cb(item): q.put(item)
    def done(): q.put(DONE)

    threading.Thread(target=start_fn, args=(cb, done)).start()

    while True:
        item = q.get()
        if item is DONE: break
        yield item

# Usage: for x in callback_to_generator(some_api): ...
```

Q74. What is the difference between a generator and a coroutine (PEP 342)?

PEP 342 enhanced generators to be full coroutines by adding `send()`, `throw()`, and `close()`. A plain generator only yields; a coroutine also receives values via `send()`. PEP 492 later introduced native `async/await` syntax.

```
# Plain generator – produces only
def producer():
    for i in range(3): yield i

# Coroutine (PEP 342) – produces AND receives
def coroutine():
    while True:
        received = yield          # receives via send()
        print('Received:', received)

c = coroutine()
next(c)
c.send(42)    # Received: 42
```

Q75. What happens if you yield inside a try/except block?

The generator suspends inside the try block. When `throw()` is called, the exception appears at the yield point and can be caught by the surrounding except. This is safe and commonly used.

```
def resilient():
    for i in range(5):
        try:
            yield i
        except ValueError as e:
            print(f' caught: {e}, continuing')

g = resilient()
print(next(g))           # 0
print(g.throw(ValueError, 'bad')) # caught: bad, continuing -> 1
print(next(g))           # 2
```

Section 4 — Performance, Patterns & Tricky Questions (Q76–Q100)

Q76. What is the performance trade-off between generators and lists?

Generators have lower memory usage and can short-circuit, but they have higher per-element CPU overhead (function call overhead per yield). For small collections or repeated iteration, a list is faster. For large/infinite streams, generators win on memory.

```
import timeit

N = 1000

list_time = timeit.timeit(lambda: sum([x**2 for x in range(N)]), number=10000)
gen_time = timeit.timeit(lambda: sum(x**2 for x in range(N)), number=10000)

print(f'List: {list_time:.3f}s')
print(f'Gen: {gen_time:.3f}s')
# List is often faster for small N due to less overhead
```

Q77. How do you profile a generator's memory usage?

Use tracemalloc to snapshot memory before and after consuming a generator vs a list.

```
import tracemalloc

tracemalloc.start()

# Generator approach
total = sum(x**2 for x in range(10**6))
snap = tracemalloc.take_snapshot()
stats = snap.statistics('lineno')
print(stats[0].size, 'bytes') # very small

tracemalloc.stop()
```

Q78. Can a generator be used with multiple consumers?

Not directly — a generator is single-pass and stateful. Use `itertools.tee()` for two consumers, or materialise to a list if you need random access or multiple passes.

```
import itertools

def source():
    yield from range(5)
```

```
g = source()
a, b = itertools.tee(g)

print(list(a))    # [0, 1, 2, 3, 4]
print(list(b))    # [0, 1, 2, 3, 4]

# Warning: tee stores seen-but-not-yet-consumed items in memory
```

Q79. What is the difference between yield and return in terms of call stack?

return pops the call frame permanently. yield suspends the frame, keeping it alive on the heap. The generator object holds a reference to this suspended frame until it is closed or garbage-collected.

```
import sys

def gen():
    frame = sys._getframe()
    print('frame id:', id(frame))
    yield
    print('resumed, same frame id:', id(sys._getframe()))

g = gen()
next(g)
# frame id: XXXXXX
next(g, None)
# resumed, same frame id: XXXXXX (same frame!)
```

Q80. What is the itertools.pairwise() function (Python 3.10+)?

pairwise(iterable) yields successive overlapping pairs: (a[0],a[1]), (a[1],a[2]), etc. Use sliding_window for earlier Python versions.

```
from itertools import pairwise    # Python 3.10+

data = [1, 2, 3, 4, 5]
print(list(pairwise(data)))
# [(1,2),(2,3),(3,4),(4,5)]

# Differences between consecutive elements
diffs = [b - a for a, b in pairwise(data)]
print(diffs)    # [1, 1, 1, 1]
```

Q81. How do you implement a generator that supports reset?

Store the initialisation parameters and provide a reset method that recreates the generator. Generators themselves cannot be rewind.

```
class ResettableRange:
    def __init__(self, n):
        self.n = n
```

```

        self._gen = self._make()

    def _make(self):
        yield from range(self.n)

    def reset(self):
        self._gen.close()
        self._gen = self._make()

    def __iter__(self): return self
    def __next__(self): return next(self._gen)

r = ResettableRange(3)
print(list(r))    # [0, 1, 2]
r.reset()
print(next(r))    # 0

```

Q82. How does Python's `sum()` use the iterator protocol?

`sum(iterable, start=0)` calls `iter()` then repeatedly calls `next()` until `StopIteration`. It works with any iterable — list, generator, or custom iterator.

```

def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

from itertools import islice
# Sum first 10 Fibonacci numbers
total = sum(islice(fib(), 10))
print(total)    # 88

```

Q83. What is the difference between `any()` and `all()` with generators?

both `any()` and `all()` short-circuit: `any()` stops at the first `True`; `all()` stops at the first `False`. Using a generator (not a list) means the rest of the sequence is never evaluated.

```

def logged_gen(it):
    for x in it:
        print('checking', x)
        yield x

print(any(x > 3 for x in logged_gen([1, 2, 5, 6])))
# checking 1  checking 2  checking 5  -> True (stops at 5)

print(all(x > 0 for x in logged_gen([1, 2, -1, 4])))
# checking 1  checking 2  checking -1  -> False (stops at -1)

```

Q84. How do you implement a generator-based observer pattern?

The subject collects generator-based observers. It sends events via `send()` to each subscribed generator.

```
def logger_observer(name):
    while True:
        event = yield
        print(f'[{name}] received: {event}')

class EventBus:
    def __init__(self): self._observers = []

    def subscribe(self, gen):
        next(gen)      # prime
        self._observers.append(gen)

    def publish(self, event):
        for obs in self._observers:
            obs.send(event)

bus = EventBus()
bus.subscribe(logger_observer('A'))
bus.subscribe(logger_observer('B'))
bus.publish('click')    # [A] received: click  [B] received: click
```

Q85. What is the difference between a generator and a class-based iterator in terms of state management?

A generator stores state implicitly in the suspended frame (local variables, instruction pointer). A class-based iterator stores state explicitly in instance attributes. Generators are more concise; class-based iterators are more flexible (multiple methods, serialisable state).

```
# Generator – state implicit
def counter(n):
    for i in range(n): yield i

# Class-based – state explicit
class Counter:
    def __init__(self, n): self.n = n; self.i = 0
    def __iter__(self): return self
    def __next__(self):
        if self.i >= self.n: raise StopIteration
        v = self.i; self.i += 1; return v

print(list(counter(3)))    # [0, 1, 2]
print(list(Counter(3)))    # [0, 1, 2]
```

Q86. How do you implement a generator-based retry mechanism?

A generator retries an operation, yielding results or sleeping between attempts — separating retry logic from business logic.

```
import time, random

def with_retry(fn, attempts=3, delay=0.1):
```

```

for attempt in range(1, attempts + 1):
    try:
        yield fn()
        return
    except Exception as e:
        print(f'Attempt {attempt} failed: {e}')
        if attempt < attempts:
            time.sleep(delay)
        raise RuntimeError('All attempts failed')

def flaky():
    if random.random() < 0.7: raise IOError('timeout')
    return 'OK'

for result in with_retry(flaky, attempts=5, delay=0):
    print('Result:', result)

```

Q87. What is `itertools.batched()` (Python 3.12+)?

`batched(iterable, n)` yields successive non-overlapping batches of length `n` as tuples. Earlier Python can implement it with `islice` in a loop.

```

# Python 3.12+
from itertools import batched
print(list(batched(range(10), 3)))
# [(0,1,2), (3,4,5), (6,7,8), (9,)]

# Backport for earlier Python
from itertools import islice
def my_batched(iterable, n):
    it = iter(iterable)
    while batch := tuple(islice(it, n)):
        yield batch

print(list(my_batched(range(10), 3)))

```

Q88. How do you implement a generator that produces chunks of a list?

Use index arithmetic in a generator to yield successive slices without copying the whole list.

```

def chunks(lst, n):
    for i in range(0, len(lst), n):
        yield lst[i:i+n]

print(list(chunks(list(range(10)), 3)))
# [[0,1,2], [3,4,5], [6,7,8], [9]]

```

Q89. What is the walrus operator (`:=`) and how does it help in iterator-heavy code?

The walrus operator assigns a value as part of an expression. In while loops with iterators it eliminates the need for a sentinel variable.


```
import io
buf = io.BytesIO(b'hello world')

# Classic
chunk = buf.read(4)
while chunk:
    print(chunk)
    chunk = buf.read(4)

# With walrus
buf.seek(0)
while chunk := buf.read(4):
    print(chunk)
```

Q90. How do you use a generator to implement a pub/sub system?

Primed coroutines act as subscribers. The publisher broadcasts by calling `send()` on each registered coroutine-generator.

```
def subscriber(name, topic):
    print(f'{name} subscribed to {topic}')
    while True:
        msg = yield
        print(f'{name} got: {msg}')

class PubSub:
    def __init__(self): self._subs = {}
    def subscribe(self, topic, gen):
        next(gen)
        self._subs.setdefault(topic, []).append(gen)
    def publish(self, topic, msg):
        for s in self._subs.get(topic, []):
            s.send(msg)

ps = PubSub()
ps.subscribe('news', subscriber('Alice', 'news'))
ps.subscribe('news', subscriber('Bob', 'news'))
ps.publish('news', 'Python 4 released!')
```

Q91. How do you write a recursive generator without stack overflow risk?

Use `yield from` for delegation. Python does not optimise tail recursion, so very deep recursion still risks `RecursionError`. Flatten the recursion iteratively using an explicit stack for very deep structures.

```
# Recursive – limited by sys.getrecursionlimit()
def recursive_flatten(lst):
    for item in lst:
        if isinstance(item, list): yield from recursive_flatten(item)
        else: yield item

# Iterative – no recursion limit
def iterative_flatten(lst):
    stack = [iter(lst)]
    while stack:
        try:
```

```

        item = next(stack[-1])
        if isinstance(item, list): stack.append(iter(item))
        else: yield item
    except StopIteration:
        stack.pop()

print(list(iterative_flatten([1, [2, [3, [4]]], 5])))    # [1, 2, 3, 4, 5]

```

Q92. What is the `__iter__` method on a generator object?

A generator object's `__iter__()` returns self. This makes every generator an iterable as well as an iterator, so you can use it directly in a for loop or pass it to any function expecting an iterable.

```

def gen(): yield 1; yield 2

g = gen()
print(iter(g) is g)    # True

# Works in for loop directly
for x in gen(): print(x)    # 1 2

# Works in sum, list, any, all, etc.
print(sum(gen()))    # 3

```

Q93. How do you implement a generator that times out?

Use signal or threading to raise an exception into the generator after a deadline, or wrap each yield with a timestamp check.

```

import time

def with_timeout(gen, seconds):
    deadline = time.monotonic() + seconds
    for item in gen:
        if time.monotonic() > deadline:
            return
        yield item

def slow_counter():
    i = 0
    while True:
        time.sleep(0.1)
        yield i
        i += 1

print(list(with_timeout(slow_counter(), 0.35)))    # [0, 1, 2]

```

Q94. How do you implement memoization for a generator?

Materialise the generator into a list the first time it is consumed, then replay from the list on subsequent calls.

```
def memoized_gen(gen_fn):
    cache = {}
    def wrapper(*args):
        if args not in cache:
            cache[args] = list(gen_fn(*args))
        yield from cache[args]
    return wrapper

@memoized_gen
def squares(n):
    print('computing...')
    for i in range(n): yield i**2

print(list(squares(4))) # computing... [0, 1, 4, 9]
print(list(squares(4))) # (no 'computing') [0, 1, 4, 9]
```

Q95. What is the difference between an iterator and a generator in CPython's implementation?

Generators are a specialised subtype of iterator with extra attributes (`gi_frame`, `gi_code`, `gi_running`, `gi_yieldfrom`). They are implemented as C-level `PyGenObject` structs. Custom class iterators are plain Python objects with `__iter__`/`__next__` methods.

```
def gen(): yield

g = gen()
print(type(g)) # <class 'generator'>
print(hasattr(g, 'gi_frame')) # True

class Iter:
    def __iter__(self): return self
    def __next__(self): raise StopIteration

it = Iter()
print(type(it)) # <class 'Iter'>
print(hasattr(it, 'gi_frame')) # False
```

Q96. How do you use a generator to implement lazy property evaluation?

Cache the generator in an attribute; on first access, advance it once and store the result. Subsequent accesses return the cached value.

```
class LazyProperty:
    def __init__(self, gen_fn):
        self.gen_fn = gen_fn
        self.name = None
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        val = next(self.gen_fn(obj))
```

```

        object.__setattr__(obj, self.name, val)    # cache
        return val

class Config:
    @LazyProperty
    def settings(self):
        print('Loading settings...')
        yield {'debug': True}

c = Config()
print(c.settings)    # Loading settings... {'debug': True}
print(c.settings)    # {'debug': True}    (cached, no reload)

```

Q97. How do async generators interact with cleanup and finally?

Async generators must be properly closed with `aclose()`. Python 3.10+ added `sys.set_asyncgen_hooks()` to run cleanup hooks automatically. Wrap in `async` with `contextlib.aclosing()` to guarantee `aclose()` is called.

```

import asyncio
from contextlib import aclosing

async def async_producer():
    try:
        for i in range(10):
            await asyncio.sleep(0)
            yield i
    finally:
        print('producer cleaned up')

async def main():
    async with aclosing(async_producer()) as gen:
        async for x in gen:
            if x == 3: break    # early exit – cleanup still runs
            print(x)

asyncio.run(main())    # 0 1 2  producer cleaned up

```

Q98. What is the inspect module's role with generators?

`inspect.isgeneratorfunction()` checks if a callable is a generator function. `inspect.isgenerator()` checks if an object is a generator iterator. Useful for decorators and frameworks that need to treat generators specially.

```

import inspect

def plain(): return 42
def gen(): yield 42

print(inspect.isgeneratorfunction(plain))    # False
print(inspect.isgeneratorfunction(gen))      # True

g = gen()
print(inspect.isgenerator(g))                # True

```

```
print(inspect.isgenerator(plain()))          # False

# Useful in decorators:
# if inspect.isgeneratorfunction(fn): wrap differently
```

Q99. How do you implement a generator-based event loop (mini asyncio)?

Use a deque of generators. Each tick, pop a generator, run it one step via `next()`. If it yields a future/task, re-queue when ready. This is the conceptual heart of asyncio.

```
from collections import deque

ready = deque()

def run(coro): ready.append(coro)

def event_loop():
    while ready:
        task = ready.popleft()
        try:
            next(task)          # run until next yield
            ready.append(task)
        except StopIteration:
            pass

def task_a():
    for i in range(3): print(f'A:{i}'); yield

def task_b():
    for i in range(2): print(f'B:{i}'); yield

run(task_a()); run(task_b())
event_loop()    # A:0 B:0 A:1 B:1 A:2
```

Q100. Putting it all together — implement a production-quality lazy pipeline.

Combine generators, itertools, contextmanager, type hints, and proper cleanup to build a robust, memory-efficient data pipeline.

```
from __future__ import annotations
from contextlib import contextmanager
from itertools import islice, chain
from typing import Iterator, Iterable, TypeVar
import time

T = TypeVar('T')

@contextmanager
def timed(label: str):
    t0 = time.perf_counter()
    try: yield
    finally: print(f'{label}: {time.perf_counter()-t0:.4f}s')

def source(n: int) -> Iterator[int]:
```

```

    yield from range(n)

def transform(it: Iterable[int]) -> Iterator[int]:
    for x in it:
        if x % 2 == 0: yield x * x

def batch(it: Iterable[T], size: int) -> Iterator[list[T]]:
    it2 = iter(it)
    while chunk := list(islice(it2, size)):
        yield chunk

def sink(batches: Iterable[list[int]]) -> int:
    total = 0
    for b in batches: total += sum(b)
    return total

with timed('pipeline'):
    pipeline = batch(transform(source(10_000)), 256)
    result = sink(pipeline)
    print(f'Sum of even squares: {result}')
# Sum of even squares: 166616670000
# pipeline: ~0.002s – O(1) memory throughout

```